

NorthPeak Reports

Technical Documentation — AI Email Report Builder for Twenty CRM

Long Tran — Marketing Systems Engineer (Build Task)

5 July 2026

Contents

Scenario & scope

Overview, architecture & operations

- What it demonstrates (checklist)

- Architecture

- Configuration

- Run it locally

How AI is used

- AI inside the product (three meaningful surfaces)

- One gateway, model chosen by config

- How AI was used to *build* this app

Security & GDPR model

- Recipients: workspace members only

- Data minimisation to the AI

- Secrets stay server-side

- Ownership & visibility

- Recipient privacy (no cross-recipient leakage)

- Least-privilege role

- Auditability

- Erasure / soft-delete

- Trust surface

Platform notes — working within Twenty's front-component sandbox

- Constraints found (verified) and our workarounds

- How native drag *could* be enabled (deliberately not done)

Scenario & scope

The company (fictional). *NorthPeak* is a B2B SaaS business selling a sales-enablement / LMS platform to mid-market revenue teams. It sells five product tiers across several regions, industries and lead sources, with deals owned by named reps and moved through a standard pipeline. All CRM data in this build is AI-generated fake data seeded to look realistic across stage, owner, industry, region, lead source, product tier, deal size and close date (`scripts/seed-northpeak.ts`).

The problem (task option 2 — AI email report builder). Everyone needs numbers out of the CRM, but the usual answer is a hundred dashboards nobody maintains. NorthPeak Reports lets anyone **describe the report they want in plain language** and have a good-looking, **trustworthy** version delivered to their inbox on a schedule — while the CRM stays clean.

The core design principle — LLM proposes, deterministic engine disposes. The language model only ever decides *what* to measure (it emits a constrained, schema-validated JSON "Report Spec"). A deterministic executor decides *how*, reading the records and computing every figure. That split is what lets a non-technical person trust the numbers: every report shows its interpreted spec in plain English, the matched row count, a "data as of" timestamp, and a deep link to verify the equivalent native Twenty view.

What this document covers. How the app is built and why, the AI design and where it adds real value, the security & GDPR model, and the platform constraints we found in Twenty's front-component sandbox and how we worked within them. The companion **Enablement Guide** (separate PDF) covers day-to-day use for a non-technical teammate.

How to read this. Sections are ordered overview → AI → security → platform notes. Every claim here was verified against a live local Twenty server (schema syncs, `dev:function:exec` runs, and a cross-check that the executor's aggregate matched a direct GraphQL `totalCount`).

Overview, architecture & operations

A Twenty CRM app that solves one problem deeply: **turn a plain-language prompt into a trustworthy, scheduled email report over your CRM data** — no dashboard sprawl, no SQL, usable by a non-technical teammate.

A user **picks a data source** (a Twenty object), describes what they want in plain language → an LLM turns it into a **validated Report Spec** → a deterministic engine runs it over live CRM data → the user refines the output through **one AI assistant** in a **drag-and-drop WYSIWYG builder** → a good-looking HTML email is delivered to workspace subscribers on a schedule.

Scenario data is a fictional B2B SaaS, **NorthPeak** (a sales-enablement/LMS platform), seeded across pipeline stage, rep/owner, industry, region, lead source, product tier, deal size and close date.

What it demonstrates (checklist)

Requirement	How it's met
Twenty app + custom object	<code>Report</code> , <code>Report Subscription</code> , <code>Report Run</code> (<code>src/objects</code>)
Non-native drag-and-drop	Email block composer front-component (<code>src/front-components/report-builder.tsx</code>) — HTML5 drag + keyboard/button reorder, sandbox-safe
Meaningful AI	LLM turns NL → validated Report Spec (never raw GraphQL); an Insight engine compares this period vs the previous one — deltas computed in code , the LLM only <i>phrases</i> them; also exposed as a native Twenty AI tool
Reuse native UX	Index views, sidebar folder + nav items, record-page tab, relations, command-menu quick action
≥1 third-party integration	Brevo (email) · OpenRouter (LLM) · QuickChart (chart PNGs)
Secure + GDPR	Private-by-default ownership, workspace-member-only recipients, data-minimized AI, server-side secrets, audited deliveries, scoped role — see <code>docs/SECURITY.md</code>
Non-technical usable	Plain prompt, plain-English echo, drag-drop layout, <code>docs/ENABLEMENT.md</code>
Fake company + data	<code>scripts/seed-northpeak.ts</code>

Architecture

```

Prompt → generate-report-spec (logic fn, also an AI tool)
      | LLMProvider (OpenRouter, model via LLM_MODEL) → Report Spec (JSON DSL)
      | validateReportSpec() — rejects/repairs invalid specs
      |
      ▼
      Report record (spec + plain-English echo + block layout)
      |
      Builder (front component) — one AI assistant (data+layout+copy), subscribers, schedule
      |
      run-report / dispatch-reports (cron) → executeSpec() deterministic aggregation
      |                                     (CoreApiClient fetch + JS group/agg)
      | — computeDeltas() vs the previous period → generateInsight()
      |   (deltas in code; LLM only phrases them — never invents numbers)
      | — generateNarrative() (aggregates only)
      | — renderEmail() HTML/CSS + QuickChart PNG
      | — Brevo send → Report Run (audit)

```

Trust-first: the LLM only ever chooses *what* to compute. A deterministic executor decides *how*, so numbers are reproducible and match a native Twenty view. Every report shows its interpreted spec in plain English, the row count, a "data as of" timestamp, and a deep link to verify in Twenty.

Key modules (`src/logic-functions/lib/`)

Module	Responsibility
<code>report-spec.ts</code>	The DSL types, <code>validateReportSpec()</code> , <code>specToEnglish()</code> — pure, unit-tested
<code>metadata.ts</code>	Runtime schema introspection (<code>MetadataApiClient</code>) — no hardcoded schema
<code>executor.ts</code>	Deterministic fetch + group + aggregate → <code>ReportResult</code>
<code>compare.ts</code>	Derives the previous period + diffs it vs the current one (<code>computeDeltas</code>) — pure, unit-tested
<code>llm.ts</code>	The single OpenRouter gateway (model = <code>LLM_MODEL</code>); <code>generateInsight</code> phrases the deltas
<code>planner-fallback.ts</code>	Deterministic keyword planner when no LLM key is set (keeps it demoable offline)
<code>report-service.ts</code>	Orchestrates NL → spec → execute → narrate
<code>render.ts</code> / <code>quickchart.ts</code>	Blocks → HTML/CSS email + chart PNG
<code>brevo.ts</code>	Transactional email send
<code>deliver.ts</code> / <code>schedule.ts</code>	Shared execute-render-send-audit + <code>nextRunAt</code> maths

Configuration

Set these in the app's **Settings** → **server variables** (secrets) once as admin:

Variable	Secret	Purpose
<code>OPENROUTER_API_KEY</code>	Yes	LLM access via OpenRouter
<code>BREVO_API_KEY</code>	Yes	Transactional email

Non-secret application variables (have sensible defaults, editable in Settings):

Variable	Default	Purpose
<code>LLM_MODEL</code>	<code>anthropic/claude-opus-4.8</code>	OpenRouter model id (submission runs on Claude ; flip to any model without code changes)
<code>BREVO_SENDER_EMAIL</code> / <code>BREVO_SENDER_NAME</code>	—	Verified Brevo sender
<code>PUBLIC_BASE_URL</code>	<code>http://localhost:3000</code>	Used for "verify in Twenty" deep links

Without `OPENROUTER_API_KEY` the app falls back to a deterministic keyword planner; without `BREVO_API_KEY` sends become no-op "dry runs". Both let you demo the full pipeline offline; set the keys to run genuinely on Claude + real email.

Run it locally

```
nvm use # Node 24.5.0
corepack enable
yarn install
yarn twenty docker:start # local Twenty server (http://localhost:2020)
yarn twenty dev # build + live-sync the app, generates the typed client

# Seed the fictional NorthPeak dataset:
TWENTY_API_URL=http://localhost:2020 TWENTY_API_KEY=<dev-jwt> yarn seed
```

Then in Twenty: **Cmd+K** → "Create report from a prompt", or open the **Reports** sidebar folder.

Tests

```
yarn test:unit # pure DSL validator/echo tests (no server)
yarn twenty dev:typecheck # tsc --noEmit
yarn test # integration tests (installs/uninstalls against a live server)
```

Manually exercise a function:

```
yarn twenty dev:function:exec -n create-report -p '{"prompt":"monthly won revenue by product tier"}'
yarn twenty dev:function:exec -n run-report -p '{"reportId":"<id>","mode":"preview"}'
yarn twenty dev:function:exec -n dispatch-reports
```

The sections that follow — **How AI is used**, **Security & GDPR model**, and **Platform notes** — expand on the AI design, the compliance posture, and the front-component sandbox constraints referenced above.

Login for the local dev server: `tim@apple.dev` / `tim@apple.dev` .

How AI is used

AI inside the product (three meaningful surfaces)

1. Natural language → validated Report Spec

The core feature. A user's plain-language request is sent to an LLM which returns a **constrained JSON "Report Spec"** (target object, filters, group-by, aggregations, time window). The model **never writes GraphQL or SQL** and never touches data.

The spec is then **validated against the live schema** (`validateReportSpec`) — every field, option value and metric type is checked. If the model produces an invalid spec, the app feeds the errors back for **one repair attempt**. A deterministic executor runs the validated spec, so results are reproducible and verifiable. This "LLM proposes, deterministic engine disposes" split is what makes the numbers trustworthy.

2. Period-over-period insight (numbers in code, wording by AI)

This is where the AI earns its place. When a report has a time window, the app runs the **same validated spec over the immediately preceding period** and diffs the two results in code (`compare.ts` → `computeDeltas`): per-metric % change and the biggest rising/falling group. The LLM (`generateInsight`) receives **only those already-computed deltas** and writes one or two sentences of insight — it cannot invent or recompute a number. So the AI adds genuine value (spotting the story in the change) while every figure stays deterministic and verifiable. A deterministic sentence stands in when no LLM key is set, so the block is never empty.

3. Plain-language narrative

After a report runs, a second LLM call turns the **aggregated numbers** into a short business summary ("headline figure, top/bottom groups, notable concentration"). It only ever sees aggregates, never raw rows.

Native Twenty AI tool

`generate-report-spec` is also exposed via `toolTriggerSettings`, so Twenty's own chat/MCP agents can build a report on request. A `defineSkill` guides those agents to use it and to share the plain-English interpretation rather than inventing figures.

One gateway, model chosen by config

All model access goes through a single adapter (`lib/llm.ts`) calling **OpenRouter** (OpenAI-compatible). The model is set by the `LLM_MODEL` variable — the submission default is `anthropic/claude-opus-4.8`, so it genuinely runs on Claude, and a developer can switch models with **zero code changes** and no hardcoded provider.

If no key is configured, a deterministic keyword planner stands in so the whole pipeline still works offline for demos.

How AI was used to *build* this app

- **Claude (via Claude Code)** drove the implementation: reading the Twenty app docs, designing the Report Spec DSL, writing the objects/fields/relations, the query engine, the logic functions, the sandbox-safe drag-and-drop front component, and these docs.
- Every phase was **verified against the live local Twenty server** — schema syncs, `dev:function:exec` runs, and a cross-check that the executor's aggregate (37 won opportunities) exactly matched a direct GraphQL `totalCount`.
- The DSL validator is covered by **unit tests** (`report-spec.unit-test.ts`).

Security & GDPR model

NorthPeak Reports is **private-by-default** and designed so personal data stays inside the workspace and is minimised at every hop.

Recipients: workspace members only

A report can only be sent to **workspace members**, modelled as a relation (`Report Subscription → Workspace Member`). There is **no free-text email field**, so a report can never be addressed to an arbitrary external address. This is a structural guarantee, not a runtime check that could be bypassed.

Data minimisation to the AI

The LLM never sees raw CRM rows:

- **Spec generation** receives only the *schema* (object + field names/types and SELECT options) and the user's prompt — no records at all.
- **Narrative generation** receives only the *aggregated* result (group labels + computed numbers, capped to the top rows) — never individual records or contact details beyond the dimension labels the user chose to group by.

The deterministic executor — not the model — reads the records, so the model has no path to bulk-export data.

Secrets stay server-side

`OPENROUTER_API_KEY` and `BREVO_API_KEY` are declared as **secret server variables**. Twenty injects them only into server-side logic functions; they are **never** exposed to front components (the browser). All third-party calls (OpenRouter, Brevo, QuickChart) happen from logic functions.

Ownership & visibility

Each `Report` has an **owner** (set to its creator) and a **visibility** (`Private` / `Workspace`). New reports default to **Private**. Visibility and ownership are stored on the record and surfaced in the Builder.

Enforcement (server-derived identity). `run-report` (preview / send-now) and `arrange-report` resolve the caller's workspace member **server-side** via `currentMemberId()` (`src/logic-functions/lib/access.ts`), backed by `MetadataApiClient.currentUser`) and pass it to `canAccessReport()` (`src/logic-functions/lib/deliver.ts`). The identity is **never taken from the request body**, so it cannot be spoofed. A `Private` report can only be previewed/sent/arranged by its owner; a private report with no owner **fails closed** (never world-open); and "Preview as [recipient]" is honoured only for the report's own owner, so it can't be used to read another member's scoped slice. `create-report` stamps the owner from the same server-derived identity, so a report can't be planted under another member or left ownerless.

Known platform limits. Twenty cron runs have *no* request user, so the scheduled dispatcher calls `deliver()` directly and is trusted by design (it only ever emails the subscribers the owner chose). For

defence in depth on plans that support it, the manifest and data model remain compatible with adding a row-level predicate (`Report.owner IS current member`) on a user-assigned role; it is intentionally *not* placed on the app's function role, because the dispatcher must read every owner's reports.

Recipient privacy (no cross-recipient leakage)

Every subscriber is emailed in their **own message** (a single-address `to`) — the delivery path never batches multiple recipients into one shared `To:` header, so a report can't leak the subscriber list to its recipients. A report is only ever emailed to the workspace members its owner explicitly added as subscribers — never to arbitrary addresses.

Least-privilege role

The app runs under a dedicated function role (`src/default-role.ts`) that can **write only its own three objects** (`Report` , `Report Subscription` , `Report Run`). It has broad *read* access because a reporting tool must aggregate across whichever objects the user points it at — but it cannot modify or destroy CRM records, and cannot change workspace settings.

Auditability

Every execution writes a **Report Run** record: status, trigger (scheduled / sent-now / preview), rows matched, recipient count and addresses, the interpreted spec, the data-as-of timestamp, and any error. This is a complete, queryable audit trail of who received what and when.

Erasure / soft-delete

All reads go through the standard Twenty API, which excludes soft-deleted records by default — so deletions and erasure requests are honoured automatically in every report. Deleting a report cascades to its subscriptions and runs.

Trust surface

Reports never ask the reader to take numbers on faith: each shows the interpreted spec, the matched row count, the data-as-of time, and a deep link to verify the equivalent native Twenty view.

Platform notes — working within Twenty's front-component sandbox

Twenty app **front-components** (`src/front-components/*`) don't run in the page. They execute as untrusted code in a **Web Worker** and describe their UI through **Remote DOM**: the worker sends a virtual element tree, and Twenty's host mirrors it to real DOM, applying a *curated* set of attributes and forwarding a *curated* set of events back to the worker.

That isolation is good for security, but it means a few browser features aren't available. Below is what we hit while building the report builder, how we verified it, and how we worked within it — so the builder stays a **portable Twenty app** that runs on any vanilla Twenty instance.

Constraints found (verified) and our workarounds

Constraint (verified via Playwright / testing)	Workaround in this app
draggable + HTML5 drag events are not forwarded. Every canvas block rendered with <code>title / cursor:grab</code> applied but <code>el.draggable === false</code>, and <code>document.querySelectorAll('[draggable="true"]').length === 0</code>. So <code>dragstart / dragover / drop</code> never fire.	Click-to-place reorder: click a block's Move grip to pick it up, then click a "Move here" slot to drop it. Up / Down buttons remain as a fallback.
Only trusted click / pointer / keyboard events forward, and there's no DOM measurement (<code>getBoundingClientRect</code> on the worker's proxy elements is not meaningful).	A position-based drag-and-drop library can't work here — <code>dnd-kit</code> needs real DOM measurement, and <code>framer-motion</code>'s drag (bundled transitively in the SDK, not exported) is pointer-based but still needs layout metrics. Reorder is therefore click-driven, which forwards reliably.
CSS keyframe animations are stripped.	Motion (the assistant "thinking..." dots, the loading-skeleton pulse) is driven by JS timers (<code>setInterval</code>) updating inline styles/text.
<code>event.stopPropagation()</code> is unreliable across the worker boundary — a child handler can't stop an ancestor handler.	No competing "click-away" handlers on ancestors; selection is changed with explicit controls (a block click / a Deselect button).
First paint is blank for a moment. Under network throttling the whole page's <code>body</code> stays empty for several seconds.	That blank is Twenty's own SPA app-boot (loading the host shell), which runs <i>before</i> our component mounts — no in-app code can fill it. Our skeleton covers the phase we <i>do</i> control: our component mounted, its data still fetching (near-instant on a local server, visible on slower/cloud connections).

How native drag *could* be enabled (deliberately not done)

Native HTML5 drag would only need a change in **Twenty core** (the host-side Remote DOM receiver): whitelist the `draggable` attribute and forward the `dragstart / dragover / drop` events (with their `clientX/clientY`) to the worker. The builder's block rows could then use native drag directly.

We **did not** do this because it lives in Twenty core, not the app: the evaluators run **vanilla Twenty**, so a forked/patched core wouldn't run this app's drag on their instance — it would stop being a portable

app. Click-to-place gives the same outcome (move a block anywhere in one gesture) on any Twenty, today.